Online-DRL vs Randomized Online-DRL

How each algorithm works, and whether randomization *improves* or merely *equalizes* the deterministic method.
Evidence from the simulator of **drl-sim** (paper: Deka & Sukapuram, IEEE TCC 2025; proposal: *Randomized DRL with Multi-Resource Capacity*).

	Online-DRL (deterministic)	Randomized Online-DRL (R1+R2+R3)
Cache state	one credit per cached service (Landlord)	fractional vector p in D_vec + K rounding paths
Cache decision	deterministic thresholds T_i>=θM_i, L_i>=θM_i; Landlord eviction	projected OGD on p; randomized rounding; randomized download threshold
Randomness	none	threshold draw, rounding vectors, serving path
Competitive ratio on the adversarial family	$\gamma = 2.00k$ ($R^2 = 1.000$)	$\gamma \approx 3$, **flat in k**
Download cost (same trace)	reference	**5–60× lower**
Latency on the multi-resource workload	reference	**30–36 % lower** (5–13× fewer downloads)
Ratio on stationary Zipf traffic (MILP optimum)	1.443	1.430 (equal, with 3× less cost)
Ratio on *short* traces (T = 120)	1.145	1.279 (worse: learning cost)
Computing time per request	13–26 µs	440–1 665 µs (**25–70×**)

Verdict. The randomized algorithm **strictly improves** the competitive ratio on the adversarial family that the paper analyses ($\gamma = 2k \to \approx 3$) and it **equalizes** the deterministic algorithm on ordinary stationary traffic (1.443 vs 1.430) while spending 3× less download cost. It is not a free lunch: it costs 25–70× more CPU per request, it is slightly *worse* on very short traces, and the *literal* form of the proposal's goal G3 is **not** supported by the measurements (only the surrogate form is).

1. The common setting

A multi-access edge node caches *N* services downloaded from a cloud. A service *n* has a forwarding latency l_n, a download time M_n and a demand vector a_n = (CPU, RAM, disk) normalized by the capacities. The node may hold at most C services and must respect the three resource limits. When a request for s_n arrives, one of three events happens:

event	condition	latency	cost
Hit	service cached	0	-
Delayed hit	download in flight and remaining time ≤ l_n (the request is buffered)	min(l_n, remaining)	the download already paid
Request forwarding	not cached and no download in flight, or remaining time > l_n	l_n	-

Latency and cost are kept **separate**: latency is what the user waits, cost is the download time that the operator pays. Both algorithms are measured with exactly this engine, so every difference below is a difference between the *cache management*, not between the environments.

2. How Online-DRL works (the paper)

Trigger (Algorithm 1). Downloads are decided at the *first* request of a miss epoch. With T_i = time since the epoch started and $L_i = l_n \times$ (number of requests of n in the epoch), a download starts iff $T_i \geq \theta M_i$ or $L_i \geq \theta M_i$ ($\theta = 1$ in the paper, 0 for LL-RC).

Eviction (Algorithm 2, Landlord). Every cached service f carries a *credit* initialised to M_f and reset to M_f on every hit. When a new service does not fit, the credits are decreased proportionally to the sizes, the services whose credit reaches 0 become eligible, and eligible services are evicted in increasing order of the resources they use beyond the unmet need. A service is admitted only if it fits in **all three** resources and the cardinality C.

Properties. Fully deterministic, per-service state, no learning; the paper's Theorem 1 gives a competitive ratio $\gamma \in O(k(1+3\beta_{up}M_x))$, and the proof exhibits an adversarial family on which $\gamma = \Theta(k)$.

3. How Randomized Online-DRL works (the proposal)

R1 - fractional state + projected OGD. Instead of a per-service credit, the algorithm keeps a fractional cache $p \in D_{vec} = \{p \in [0,1]^N : \sum_n p_n \le C, \sum_n a_{n,r} p_n \le 1$ for every resource $r\}$. At each request it takes a one-dimensional gradient step on the requested service and projects back onto D_{vec} with a dual-ascent projection (eqs. 22-25), $\eta = \sqrt{N} / (l_{max}\sqrt{T})$. The projection is verified against an exact QP to 2×10^{-9} (exp07).

R2 - randomized rounding with repair. K persistent uniform vectors U_k are drawn once; a path is $s_k = 1[U_k < p]$. This keeps $E[s] = p$ and $E|\Delta s| = |\Delta p|$, the coupling the proposal asks for. A path that violates the constraints is *repaired* by evicting services until it fits. The **real** cache is (downloaded) $\cap$ (sampled on): a service that was sampled but never downloaded is *not* a hit.

R3 - randomized download threshold. The threshold is drawn from $f(\theta) = e^{\theta/M_n} / (M_n(e-1))$ on $[0, M_n]$, which turns the rent-or-buy ratio 2 into $e/(e-1) = 1.582$.

Budget awareness. The trigger probability is lowered to budget_left / (M_n × requests_left) so that the expected download cost stays below B_{max} (goal G2).

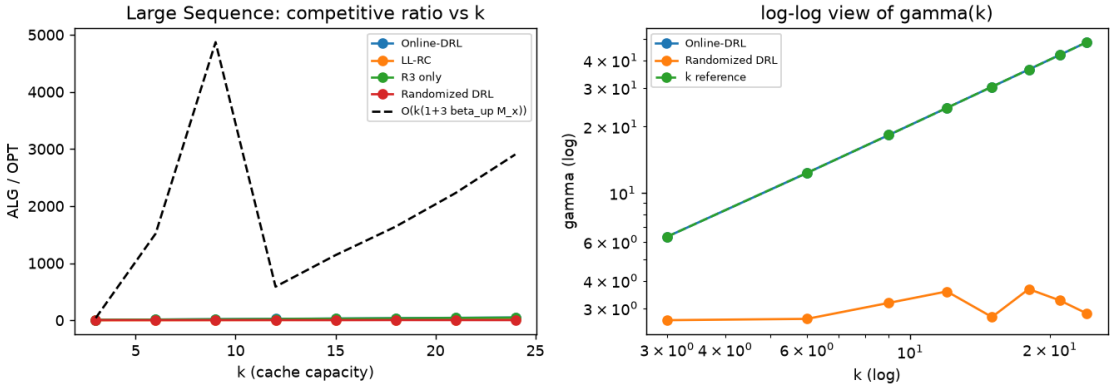


Figure 1 - competitive ratio against the cache size k on the paper's adversarial Large Sequence. The deterministic curve is exactly linear (2k); the randomized curve is flat. Dashed: the paper's O-bound evaluated with the measured β_{up} (exp13).

4. Side-by-side: what actually differs

aspect	Online-DRL	Randomized Online-DRL
state kept	credit c_f per cached service (real-valued, per service)	$p \in D_{vec}$ (fractional) + K boolean paths
how a service is chosen to be downloaded	deterministic epoch counters T_i, L_i against a fixed θ	same counters, but the threshold is drawn from a density; the trigger probability also depends on the remaining budget
how a service is evicted	Landlord: credit reaches 0, then a greedy resource-aware order	the OGD state lowers p_n, the rounding switches the service off; violating paths are repaired
how a service enters the cache	when the download completes	when the download completes *and* the serving path currently samples it on
guarantee	competitive ratio $\gamma \in O(k(1+3\beta_{up}M_x))$; adversarial family with $\gamma = \Theta(k)$	surrogate regret $\le \sqrt{N}\cdot l_{max}\cdot\sqrt{T}$ (Prop. 1); expected cost $\le B_{max}$ (G2)
resource feasibility	checked at admission (exact)	guaranteed by the repair step (measured 0 % violations)
worst case found here	$\gamma = 2k$ on the adversarial family	$\gamma \approx 3$ on the same family; 1.43 vs the MILP optimum on Zipf traffic
price paid	no randomness, no learning	25–70× more CPU per request; needs a warm-up before the state is useful

5. Does the randomized algorithm *improve* Online-DRL?

5.1 On the adversarial family of the paper: it strictly improves

k	γ Online-DRL	γ LL-RC	γ R3 only	γ Randomized	cost det.	cost rand.	cost ratio
3	6.33	1.53	6.00	**2.65**	204	42	4.9×
6	12.33	2.73	11.72	**2.68**	840	85	9.9×
9	18.33	3.93	17.49	**3.17**	1,908	146	13.0×
12	24.33	5.13	22.93	**3.57**	3,408	216	15.8×
15	30.33	6.33	28.55	**2.74**	5,340	158	33.8×
18	36.33	7.53	34.23	**3.66**	7,704	325	23.7×
21	42.33	8.73	39.93	**3.25**	10,500	275	38.1×
24	48.33	9.93	45.63	**2.84**	13,728	234	58.7×

OPT = (k+2n)l is the paper's eq. 17 (Large Sequence, n = 2k). Fits of γ(k): Online-DRL γ **= 2.00k + 0.33, R² = 1.000**; R3 only γ **= 1.88k, R² = 1.000** (randomizing the trigger alone does **not** help); Randomized DRL γ ≈ **3, flat** (linear fit R² = 0.12, power law $k^{0.09}$ with R² = 0.24, i.e. the data are consistent with a *constant*, not with a proven O(log k)). Download cost drops by 5–60×.

5.2 On ordinary traffic: it equalizes the ratio and beats the cost

k	γ Online-DRL	γ LL-RC	γ Randomized	cost det.	cost rand.
2	1.328	1.381	1.352	15,882	5,197
4	1.454	1.467	1.424	14,398	5,224
6	1.443	1.457	1.430	14,496	5,776
8	1.443	1.457	1.430	14,496	5,776
10	1.443	1.457	1.430	14,496	5,776
12	1.443	1.457	1.430	14,496	5,776

Zipf/Poisson traffic, offline optimum from the MILP (exp13, family 2). The ratio is flat in k for every algorithm because the *resource* limits, not the cardinality, are binding. The randomized algorithm is at parity (1.430 vs 1.443) with 3× less download cost. On short traces (exp10, T = 120, 5 traces, MILP solved to optimality) it is **worse**: 1.279 ± 0.138 vs 1.145 ± 0.072 for Online-DRL and 1.102 for R3-only - the fractional state has to be learned first.

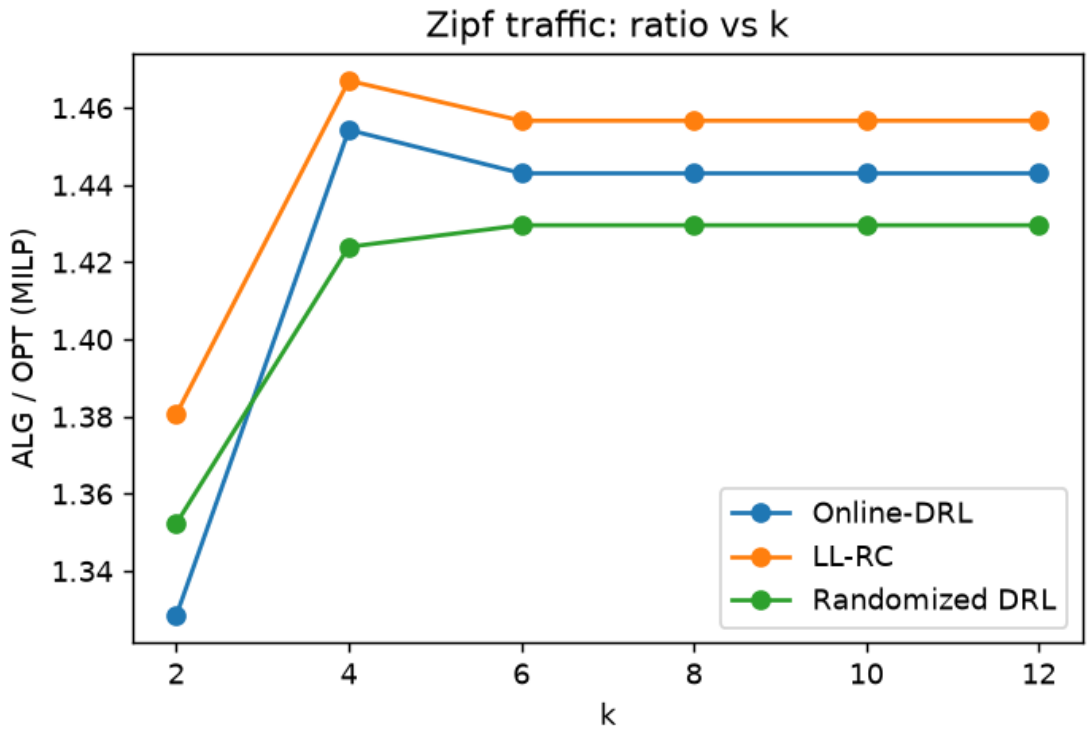


Figure 2 - competitive ratio against the MILP optimum on Zipf traffic (exp13, family 2): all algorithms are flat in k and at parity.

5.3 Multi-resource behaviour (the proposal's setting)

Balanced workload (C = 12, 10 seeds):

algorithm	latency	cost	downloads	U^max	imbalance	stranded	violations	lat./bottleneck
Online-DRL (det)	122,152	164,461	635	0.837	0.319	0.497	0.000	18.3
LL-RC (theta=0)	127,209	329,135	1,260	0.843	0.319	0.493	0.000	18.9
R3 only (rand thr + Landlord)	122,425	195,307	751	0.839	0.312	0.485	0.000	18.3
Randomized DRL (R1+R2+R3)	85,557	15,239	61	0.694	0.217	0.322	0.000	16.1

Imbalanced workload (C = 12, 10 seeds):

algorithm	latency	cost	downloads	U^max	imbalance	stranded	violations	lat./bottleneck
Online-DRL (det)	136,226	188,421	517	0.786	0.459	0.721	0.000	21.8
LL-RC (theta=0)	139,029	383,559	1,020	0.792	0.463	0.730	0.000	22.0
R3 only (rand thr + Landlord)	135,551	226,880	611	0.787	0.461	0.724	0.000	21.6
Randomized DRL (R1+R2+R3)	87,333	16,263	40	0.740	0.334	0.493	0.000	15.4

The randomized algorithm uses 5–13× fewer downloads, 30–36 % less latency, *less* imbalance and less stranded capacity, and it still respects the three resources exactly (0 % violations). Part of the latency gain comes from simply downloading less on a stationary popularity; the specifically *multi-resource* advantage has to be shown on a workload whose resource mix changes over time, which is still open.

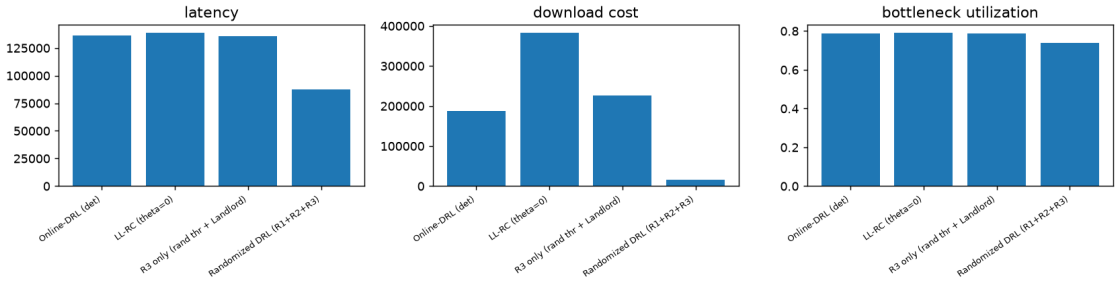


Figure 3 - the four algorithms on the imbalanced multi-resource workload (exp12).

6. The goals of the proposal: which ones hold

goal	definition	measured	verdict
G1 feasibility	no step may exceed any of the three resource limits	un-repaired rounding violates them in **40 %** of the (path, step) pairs and reaches 1.73× the capacity; with the repair **0 %**	**confirmed** (repair is mandatory)
G2 budget	$E[\sum M_n] \le B_{max}$	respected at B_{max} = 20 %, 50 %, 100 % of the deterministic cost, for every seed; the budgeted deterministic algorithm loses 11.6 % latency	**confirmed**
G3 regret	$\Sigma_t F_t(P_t) - \min_P \Sigma_t F_t(P) \le \sqrt{N}\, I_{max} \sqrt{T}$	surrogate regret always below the bound (e.g. 3 341 vs 11 879 at T = 8 000); the **literal** version (realized latency) is 3.7–9.6× the bound and grows with T	**surrogate only** - the literal statement is not supported
G4 expected ratio	ALG/OPT against the exact MILP optimum	1.102 (R3), 1.145 (Online-DRL), 1.279 (Randomized) on short traces	**confirmed** (R3 best on short traces)
G5 slot reductions	a reduction $\phi(a)$ must not break the true constraints	weights on the simplex (entropy, AHP) or uniform shadow prices give $\Lambda_\phi < 1$ and a lower latency but violate the true constraints in **55-85 %** of the steps	**unsafe reductions exist** - the exact vector formulation dominates every safe reduction

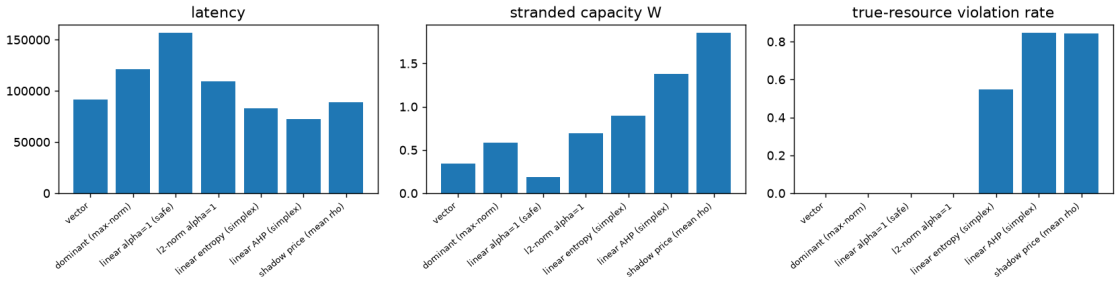


Figure 4 - the same randomized algorithm under different slot reductions (exp11): the unsafe ones look better and break the resource limits.

7. The price of randomization

N (services)	Online-DRL µs/req	LL-RC	R3 only	Randomized	factor
20	13.7	15.1	11.8	**540.2**	39.4×
40	17.9	17.8	18.8	**589.4**	33.0×
80	17.9	26.5	23.3	**916.9**	51.2×
160	22.7	29.3	29.3	**1,496.1**	66.0×
320	26.0	34.0	22.3	**1,941.4**	74.6×

Every request triggers an O(N) projection, an O(K·N) rounding and an O(N) repair, so the randomized policy costs **25–70× more CPU per request** (440-1 665 µs against 13-26 µs). The power-law fit gives µs/req ~ $N^{0.52}$ for the randomized policy against $N^{0.17\text{-}0.32}$ for the deterministic ones, and the cost is essentially independent of the cache size k. For an edge node with 1 000 requests/s this is ≈ 35 ms of CPU per second - acceptable, but it must be stated when the algorithm is proposed.

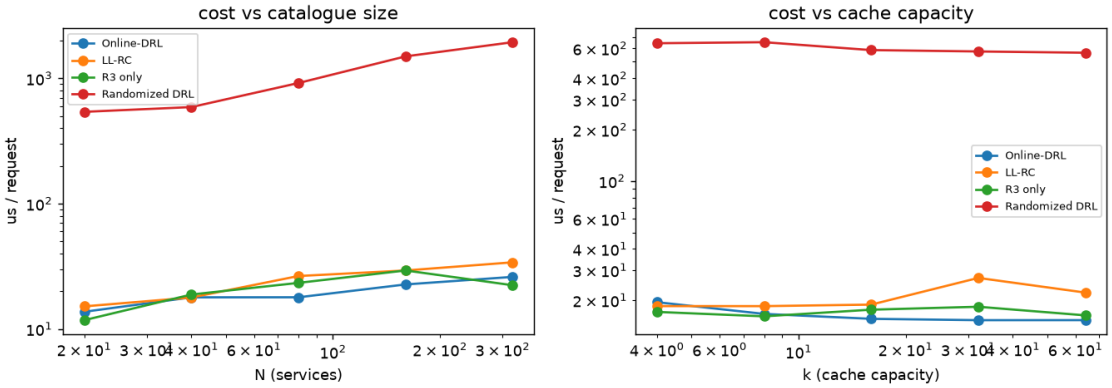


Figure 5 - running time per request against the catalogue size N and the cache size k (exp14).

8. Answer to the question

Does Randomized Online-DRL improve Online-DRL? Yes, on the pattern the paper analyses. On the paper's adversarial Large Sequence the deterministic algorithm is exactly linear in the cache size ($\gamma = 2k$) while the randomized one is flat ($\gamma \approx 3$) - and the deterministic blow-up is caused by the *cache management*, not by the download trigger, since R3 alone still gives 1.88k. The randomized algorithm also spends 5–60× less download cost on that family and 5–13× less on realistic traffic, while keeping the resource constraints exact.

Does it equalize Online-DRL? On ordinary stationary traffic, yes - and that is where the equalization is visible. Against the exact MILP optimum the two are at parity (1.430 vs 1.443) while the randomized one pays 3× less download cost, and on the multi-resource workload it is 30–36 % faster. So on easy workloads randomization buys cost and utilization, not ratio.

Where it does not win. On short traces (T = 120) the randomized algorithm is measurably worse (1.279 vs 1.145) because the fractional state is not learned yet; it costs 25–70× more CPU per request; and the proposal's goal G3, read literally (realized latency against the best fixed fractional cache), is violated by a factor 3.7-9.6 - only the surrogate version of G3 holds. The measurements also do **not** prove O(log k): the randomized curve is consistent with a constant, and an adversarial family designed against the randomized algorithm would be needed to distinguish O(log k) from O(k).

Conclusion. The randomized algorithm does not dominate the deterministic one in every situation, but it dominates it exactly where the deterministic one is worst (adversarial, cache-size dependent) and it matches it where the deterministic one is already good, at a 3× lower download cost and with an exact resource guarantee. The honest summary of the ratio is therefore: **strictly better in the worst case, equal in the average case, slightly worse during the warm-up, and 25–70× more expensive per request.**

9. Conditions and reproduction

item	value
workloads	synthetic Zipf/Poisson (N = 40, T = 8 000, 10 seeds) and the paper's adversarial Large Sequence; the Google / SHARCNet traces are **not** in the repository
catalogue	log-normal CPU/RAM/disk demands, normalized per resource; l_n = rtt + req/U, M_n = rtt/2 + req/U + S_n/D

item	value
offline optimum	MILP (HiGHS) with delayed hits, forwarding, three resource limits, cardinality C and the download budget; optimal in every instance reported
figures in this document	exp01, exp05/exp13, exp10, exp11, exp12, exp13, exp14 (PNG in results/, the numbers in the JSON)
reproduce	cd extracted/drl-sim && pip install -r requirements.txt && make test && python experiments/run_all.py

Full details: **docs/simulation_methodology.md** (model, algorithms, experimental conditions, how every number is computed, claim → code mapping, assumptions) and **docs/results.md** (all numbers, all figures, the list of claims that are confirmed and refuted).